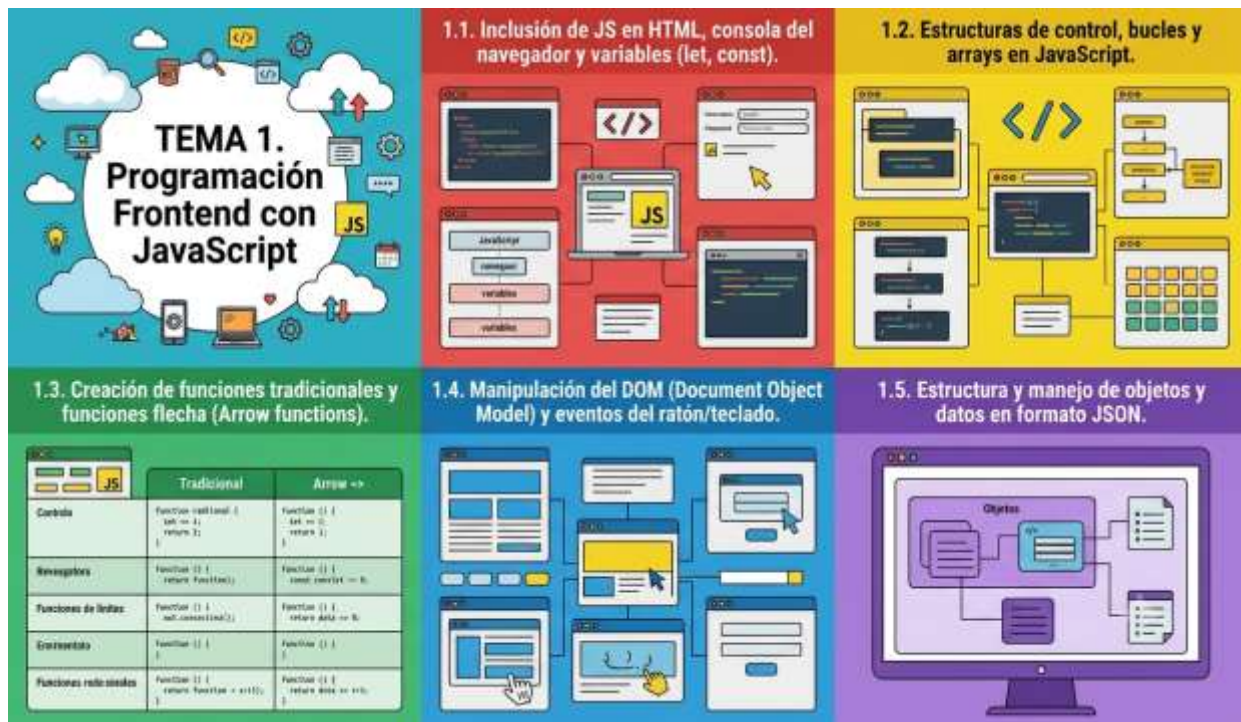


TEMA N° 1



TEMA 1 PROGRAMACIÓN FRONTEND CON JAVASCRIPT



EN ESTE TEMA SERÁS CAPAZ DE...

1. **Dinamizar interfaces web:** Aprenderás a dar vida a las páginas estáticas usando JavaScript, conectando las acciones del usuario con respuestas visuales inmediatas.
2. **Estructurar y manipular datos locales:** Dominarás el uso de variables, arrays y objetos JSON para gestionar información del mundo real, aplicable a los sistemas que desarrolles en tu entorno.
3. **Prepararte para la web moderna y la IA:** Comprenderás desde la manipulación del DOM hasta los conceptos avanzados que permiten a un Técnico en Sistemas Informáticos integrar herramientas modernas como el consumo de APIs y servicios de Inteligencia Artificial directamente en el navegador.



PRÁCTICA

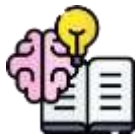


¿Cómo podemos transformar una página estática, que solo muestra información, en una herramienta viva que reaccione a cada clic y decisión de nuestros usuarios?

Imagina la siguiente situación en nuestra comunidad: El flamante y bien equipado módulo educativo del CEA Herman Ortiz Camargo, ubicado aquí en el Barrio Saó, acaba de lanzar una campaña masiva de inscripción técnica para los jóvenes de Pailón. Desde la dirección te han pedido crear una página web donde los estudiantes puedan registrarse.

Sin embargo, hay un problema logístico. Se necesita organizar a los estudiantes para sus prácticas dependiendo del barrio donde viven. Por ejemplo, los proyectos de comercio se asignarán a los inscritos del **Barrio Central Fátima**, los proyectos de recreación deportiva a los del **Barrio 24 de Septiembre** y **Barrio 27 de Mayo**, los de espacios verdes al **Barrio Residencial Norte**, y los de desarrollo comunitario a barrios como **María Belén, San Antonio, Jardines de Pailón y San Expedito**.

Actualmente, la página solo tiene un diseño en HTML y CSS. Cuando un usuario hace clic en el botón "Registrar" o intenta filtrar los barrios, no sucede absolutamente nada. La página es "sorda y muda" ante las interacciones del usuario. Como Técnico en Sistemas Informáticos, tu desafío es darle "cerebro" a esta interfaz para que al seleccionar un barrio, el sistema automáticamente asigne y muestre en pantalla el lugar del proyecto, guarde esa información y confirme el registro al estudiante. ¡Aquí es donde entra JavaScript!



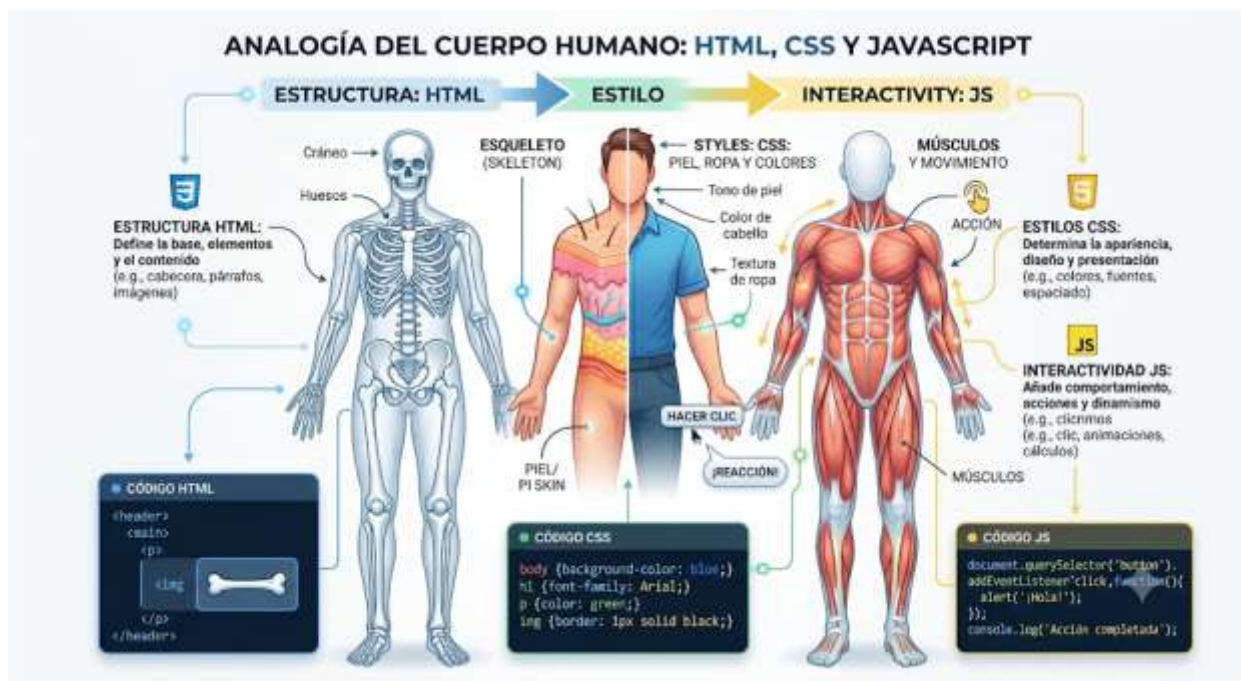
TEORÍA



JavaScript (JS) es el lenguaje de programación que da interactividad a la web. Mientras el HTML da la estructura y el CSS el diseño, JS es el "músculo" que permite realizar acciones.

1.1. INCLUSIÓN DE JS EN HTML, CONSOLA DEL NAVEGADOR Y VARIABLES (LET, CONST)

Para que nuestro código JavaScript funcione, debemos conectarlo con nuestro archivo HTML. Existen varias formas, pero la mejor práctica es usar un archivo externo.



Cómo incluir JS en HTML:

Se utiliza la etiqueta `<script>` justo antes de cerrar la etiqueta `</body>` en tu HTML.

HTML

```
<!DOCTYPE html>
<html lang="
es">
<head>   <meta charset="
UTF-8">   <title>Inscripciones CEA Herman Ortiz Camargo</title>
```



```
</head>
<body>    <h1>Bienvenido al CEA</h1>        <script src="
logica.js">
</script>
</body>
</html>
```

La Consola del Navegador:

Es tu mejor amiga como Técnico. Presionando F12 (o clic derecho -> Inspeccionar -> Consola) en tu navegador, puedes probar código en vivo y ver errores. Usamos `console.log()` para imprimir mensajes allí.

Variables (let y const):

Una variable es una "caja" donde guardamos datos. En el JavaScript moderno usamos `let` para valores que van a cambiar y `const` para valores que serán fijos.

```
JavaScript

// Usamos '
const
' porque el nombre del CEA no va a cambiarconst nombreInstitucion = "
CEA Herman Ortiz Camargo";
// Usamos '
let
' porque la cantidad de estudiantes registrados puede aumentarlet
estudiantesRegistrados = 0;
// Imprimimos en la consola para verificar
// El símbolo + une (concatena) texto con variablesconsole.Log("
Bienvenidos al " + nombreInstitucion);
```

1.2. ESTRUCTURAS DE CONTROL, BUCLES Y ARRAYS EN JAVASCRIPT

Para que un programa tome decisiones o repita tareas, necesitamos estructuras de control y formas de agrupar datos.

Arrays (Arreglos):

Son listas de datos guardados en una sola variable. Los índices (posiciones) empiezan en 0.

```
JavaScript

// Creamos un array (Lista) con algunos barrios de Pailón
// El índice 0 es '
```



```

Saó', el 1 es '
San Antonio', etc.const barriosPailon = [
Saó', '
San Antonio', '
Ovidio Barberi', '
Las Misiones'];
// Accedemos al primer barrio de la lista e imprimimos en
console.log(barriosPailon[0]);
// Mostrará: Saó

```

Estructuras de control (if / else):

Permiten ejecutar un código solo si se cumple una condición.



JavaScript

```

// Variable que guarda el barrio seleccionado por el usuariolet
barrioSeleccionado = '
Central Fátima';
// Evaluamos si el barrio es '
Central Fátima'
if (barrioSeleccionado === '
Central Fátima') {
// Si es verdadero, ejecutamos esta línea console.log("
Proyecto asignado: Comercio formal e Iglesia");
}
else {
// Si es falso (cualquier otro barrio), ejecutamos esta línea console.log("

```



```
Proyecto asignado: Desarrollo general");  
}
```

Bucles (for):

Repiten un bloque de código varias veces, ideal para recorrer nuestros Arrays.

JavaScript

```
// Bucle for: // 1. Let i = 0 (empezamos en el índice 0)  
// 2. i < barriosPailon.length (repetimos mientras i sea menor a la cantidad de  
barrios)  
// 3. i++ (sumamos 1 a i en cada vuelta)  
for (let i = 0;  
i < barriosPailon.length;  
i++) {  
// Imprimimos cada barrio, uno por uno console.log("  
Barrio registrado en el sistema: " + barriosPailon[i]);  
}
```

1.3. CREACIÓN DE FUNCIONES TRADICIONALES Y FUNCIONES FLECHA (ARROW FUNCTIONS)

Las funciones son bloques de código que guardamos para usarlos (llamarlos) cada vez que los necesitemos, sin repetir código.

Función Tradicional:

JavaScript

```
// Declaramos una función llamada '  
saludarEstudiante  
' que recibe un parámetro '  
nombre'  
function saludarEstudiante(nombre) {  
// Retorna (devuelve) un mensaje combinando texto y la variable return "  
Hola " + nombre + ", bienvenido al módulo educativo del CEA."  
}  
// Llamamos a la función y guardamos su resultado en una variableconst mensaje =  
saludarEstudiante("  
Carlos");  
// Imprimimos el resultadoconsole.log(mensaje);
```

Funciones Flecha (Arrow Functions):

Es una forma más moderna, corta y limpia de escribir funciones. Son muy usadas en el desarrollo actual.

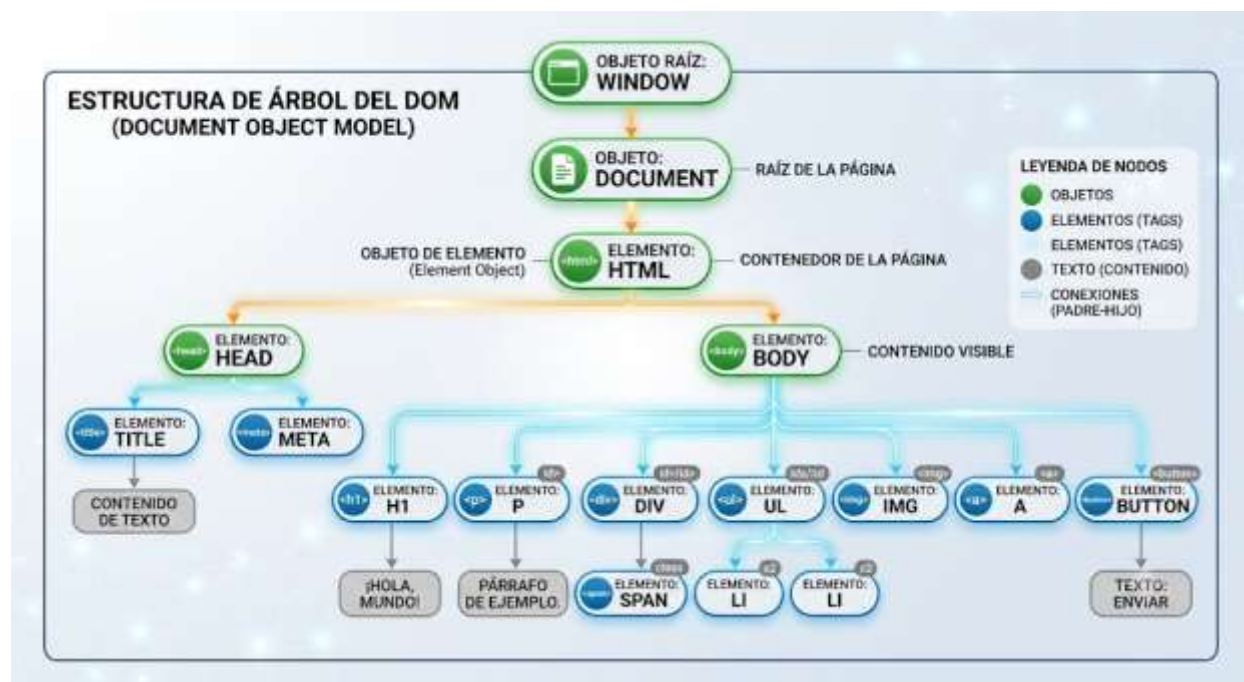


JavaScript

```
// Declaramos una constante que almacena la función flecha
// (nombre, barrio) son los parámetros de entrada
// => indica que es una función flecha
const asignarProyecto = (nombre, barrio) => {
  // Usamos backticks (`) para crear plantillas literales (Template Literals)
  // Permiten inyectar variables usando {variable}
  más fácilmente   return `El estudiante {
  nombre}
  del barrio {
  barrio}
  ha sido registrado.`;
}
// Llamamos a la función flecha
console.log(asignarProyecto("
Ana", "
San José"));
```

1.4. MANIPULACIÓN DEL DOM (DOCUMENT OBJECT MODEL) Y EVENTOS DEL RATÓN/TECLADO

El DOM es la representación que hace el navegador de nuestro HTML. JavaScript puede "agarrar" cualquier elemento del HTML y modificarlo en tiempo real.



**Eventos:**

Son acciones que ocurren en la página (un clic, presionar una tecla, mover el ratón).

HTML

```
<button id="
btnRegistrar">Registrar en el CEA</button>
<p id="
mensajeConfirmacion">
</p>
```

JavaScript

```
// 1. '
Agarramos
' el botón de HTML usando su ID y lo guardamos en una constanteconst boton =
document.getElementById("
btnRegistrar");
// 2. '
Agarramos
' el párrafo vacío donde mostraremos el mensajeconst parrafoMensaje =
document.getElementById("
mensajeConfirmacion");
// 3. Le agregamos un '
escuchador de eventos
' (addEventListener) al botón
// Escuchamos el evento '
click', y cuando ocurra, ejecutamos la función flechaboton.addEventListener("
click", () => {
// Cambiamos el texto interno (textContent) del párrafo
parrafoMensaje.textContent = "¡Registro exitoso! Te esperamos en el Barrio
Saó.";
// También podemos cambiar el CSS desde JS parrafoMensaje.style.color = "
green";
}
);
```

1.5. ESTRUCTURA Y MANEJO DE OBJETOS Y DATOS EN FORMATO JSON

Mientras los arrays son listas simples, los **Objetos** permiten guardar datos estructurados usando pares de clave: valor.



JavaScript

```
// Creamos un objeto para representar a un estudiante con múltiples
propiedadesconst estudianteCEA = {
  nombre: "
  Luis Alberto", // Clave: '
  nombre', Valor: '
  Luis Alberto
  '    edad: 22,           // Clave: '
  edad', Valor: 22    barrio: "27 de Mayo", // Clave: '
  barrio', Valor: '27 de Mayo
  '    activo: true       // Clave: '
  activo', Valor: true (booleano)}
;
// Accedemos a las propiedades usando un punto (.)
console.log("
Nombre del alumno: " + estudianteCEA.nombre);
```

JSON (JavaScript Object Notation):

Es el formato universal para enviar y recibir datos en la web. Es idéntico a los objetos de JS, pero escrito estrictamente como texto (las claves llevan comillas dobles).

JavaScript

```
// Convertimos el objeto de JavaScript a un texto (String) en formato JSON
// Esto es útil para enviar datos por internetconst estudianteJSON =
JSON.stringify(estudianteCEA);
console.log(estudianteJSON);
// Imprime el objeto pero como un texto plano
// Convertimos un texto JSON de vuelta a un objeto real de JavaScript
// Útil cuando recibimos datos de internet y queremos usarlosconst
estudianteRecuperado = JSON.parse(estudianteJSON);
console.log(estudianteRecuperado.barrio);
// Vuelve a imprimir '27 de Mayo'
```

1.6. CONSUMO DE APIS WEB (FETCH) Y PROMESAS (CONCEPTO AVANZADO)

Como Técnico, no solo crearás sistemas aislados, sino que conectarás tu interfaz con servidores y bases de datos a través de APIs (Interfaces de Programación de Aplicaciones). Al hacer esto, la respuesta no es inmediata, toma unos milisegundos. Para ello usamos "Promesas" y la función `fetch()`.



JavaScript

```
// Hacemos una petición (Fetch) a una API pública para obtener datos de usuarios de prueba
// fetch() devuelve una Promesa (Promise) de que entregará algo en el futuro
fetch('https://jsonplaceholder.typicode.com/users')
    // .then() captura la respuesta cuando llegue y la convierte a formato JSON
    .then(respuesta => respuesta.json())
    // El siguiente .then() recibe los datos ya convertidos y los usa
    .then(datos => {
        // Imprimimos el primer usuario que nos devolvió internet
        console.log("Datos recibidos de internet:", datos[0].name);
    })
    // .catch() atrapa y maneja cualquier error (ej. sin conexión a internet)
    .catch(error => console.log("Error al conectar:", error));
```

1.7. ALMACENAMIENTO LOCAL EN EL NAVEGADOR (LOCALSTORAGE) (CONCEPTO AVANZADO)

Muchas veces necesitamos que, si el estudiante cierra el navegador, no pierda su progreso. localStorage permite guardar información en el equipo del usuario de Pailón sin necesitar aún una base de datos externa.

JavaScript

```
// Guardamos un dato en la memoria del navegador del usuario
// Usamos setItem y le damos un nombre ('barrioUsuario') y un valor ('Oto Waver')
localStorage.setItem('barrioUsuario', 'Oto Waver');
// Recuperamos ese dato de la memoria usando getItem
// Lo guardamos en una constante para usarlo
const barrioGuardado = localStorage.getItem('barrioUsuario');
// Lo imprimimos para comprobar que se guardó
console.log("El último barrio visitado fue: " + barrioGuardado);
// Si cierras el navegador y vuelves a abrirlo, este dato seguirá existiendo
```



1.8. INTEGRACIÓN DE IA Y TENDENCIAS EN EL FRONTEND (CONCEPTO AVANZADO)

El rol del Técnico en Sistemas Informáticos está evolucionando. Hoy, el Frontend no solo muestra datos, sino que interactúa con Modelos de Lenguaje (LLMs) como ChatGPT o Gemini a través de APIs para generar contenido dinámico.

Imagina un sistema en el CEA donde, al inscribirse, una IA analiza las preferencias del estudiante y le sugiere instantáneamente la mejor carrera técnica. Usando la lógica de "Fetch" (explicada en el 1.6), puedes enviar el JSON de las habilidades del estudiante (explicado en el 1.5) a una API de Inteligencia Artificial, recibir su respuesta, y manipular el DOM (explicado en el 1.4) para mostrarla en pantalla. ¡Ese es el poder del desarrollo moderno!

CIERRE TEÓRICO OBLIGATORIO

❑ Mitos vs. Realidades

Mito	Realidad
"JavaScript y Java son el mismo lenguaje."	Totalmente falso. Comparten parte del nombre por razones de marketing en los años 90. Tienen propósitos, sintaxis y usos completamente distintos.
"JS solo sirve para poner animaciones en la web."	Falso. Hoy en día JS es un lenguaje de propósito general. Con tecnologías como Node.js, puede gestionar servidores, bases de datos e incluso crear aplicaciones móviles.

❑ Glosario de Términos

1. **DOM:** Representación en forma de árbol de los elementos de una página web, modificable por JS.
2. **JSON:** Formato ligero de intercambio de datos, fácil de leer para humanos y máquinas.
3. **Array:** Colección ordenada de datos almacenada bajo una sola variable.
4. **Fetch:** Función de JavaScript que permite realizar peticiones de red para obtener recursos de internet.



5. **Evento:** Interacción o suceso que ocurre en el navegador (como un clic) que JS puede detectar para reaccionar.

□ **Ponte a Prueba**

1. ¿Qué palabra clave usamos hoy en día para declarar una variable cuyo valor no cambiará?
 - a) var
 - b) let
 - c) const
2. ¿Qué estructura de datos es ideal para guardar pares de "clave: valor", como los datos de un estudiante?
 - a) Array
 - b) Objeto
 - c) Bucle for
3. Para agarrar un elemento HTML con el id "titulo" desde JS usamos:
 - a) document.getElementById("titulo")
 - b) document.agarrar("titulo")
 - c) HTML.getElement("titulo")

(Respuestas correctas: 1. c, 2. b, 3. a)



VALORACIÓN



Como futuros profesionales del CEA Herman Ortiz Camargo, es vital comprender que programar no es solo teclear código; es crear herramientas con impacto en nuestra sociedad pailoneña.

1. **Impacto Social y Privacidad:** Al recolectar datos de nuestros vecinos en el sistema de inscripción, ¿estamos garantizando que esta información (nombres, números de teléfono, direcciones en barrios como Jardines de Pailón o María Belén) no caiga en malas manos? Manipular correctamente los objetos y enviar datos seguros por la red es tu responsabilidad ética.



2. **Inclusión Digital:** Una interfaz rápida e intuitiva lograda con un buen manejo del DOM y JavaScript permite que personas sin mucha experiencia tecnológica, como comerciantes mayores de Fátima o agricultores locales, puedan usar tu sistema sin frustrarse.
3. **E-waste (Basura Electrónica):** Aplicaciones web mal optimizadas, con bucles infinitos o código pesado, consumen excesiva batería y procesador. Esto acorta la vida útil de computadoras y teléfonos antiguos, obligando a los usuarios a desecharlos prematuramente y generando contaminación ambiental. Escribir código limpio y eficiente ayuda a la ecología.



PRODUCCIÓN



Reto Final (Laboratorio): El Selector Inteligente del CEA

Objetivo: Desarrollarás una mini-aplicación funcional para el módulo del CEA en el Barrio Saó. Cuando un usuario elija su barrio de residencia, la pantalla debe indicarle automáticamente en qué área realizará sus prácticas.

Paso a Paso:

1. Crea el HTML (index.html):

1. Crea una lista desplegable (`<select id="listaBarrios">`) con opciones (`<option>`) para: "Central Fátima", "24 de Septiembre", "Residencial Norte", "Saó".
2. Añade un botón: `<button id="btnAsignar">Ver Área de Práctica</button>`.
3. Añade un párrafo vacío: `<p id="resultado"></p>`.
4. No olvides enlazar tu JS al final: `<script src="app.js"></script>`.

2. Crea el JavaScript (app.js):

1. Declara tres variables constantes (const) seleccionando el select, el button y el p utilizando `document.getElementById()`.
2. Agrega un evento click al botón con `addEventListener`.



3. Dentro del evento, obtén el valor actual de la lista desplegable usando `listaBarrios.value`.
4. Usa una estructura `if / else if` o un `switch` para evaluar:
 1. Si es "Central Fátima" -> El párrafo debe decir "Área de prácticas: Comercio y Administración".
 2. Si es "24 de Septiembre" -> El párrafo debe decir "Área de prácticas: Proyectos Deportivos".
 3. Si es "Residencial Norte" -> "Área de prácticas: Parques y Espacios Verdes".
 4. Si es "Saó" -> "Área de prácticas: Módulo Principal CEA".
5. Cambia el color del texto del párrafo usando la propiedad `style.color` a tu color favorito para dar retroalimentación visual de éxito.

3. Prueba y Depuración:

1. Abre el archivo HTML en tu navegador. Presiona F12 y abre la Consola.
2. Interactúa con tu página. Si el texto no cambia, revisa la consola buscando errores de escritura o variables no definidas. ¡Recuerda que el Técnico resuelve problemas observando las pistas!

RECURSOS ADICIONALES

Para seguir expandiendo tus conocimientos como Técnico en Sistemas Informáticos, te recomiendo consultar la siguiente documentación oficial y gratuita:

1. MDN Web Docs (Mozilla Developer Network) - JavaScript:

La "biblia" del desarrollador web. Tiene la documentación más precisa y tutoriales desde cero.

Enlace sugerido de búsqueda: MDN JavaScript Guía

2. JavaScript.info:

Un manual moderno, detallado y paso a paso que cubre todo, desde variables simples



hasta Promesas y consumo de APIs.

Enlace sugerido de búsqueda: es.javascript.info

3. **GitHub / CodePen:**

Crea una cuenta en CodePen para practicar tu HTML, CSS y JS directamente en el navegador, y en GitHub para guardar los códigos que vayas desarrollando en tu formación en el CEA.